

CSS Z-Index: Complete Guide

CSS Learning Resources

May 18, 2026

Contents

1 Introduction

1.1 Why Z-Index Matters

2 Understanding Z-Index Fundamentals

2.1 Basic Syntax

2.2 Key Rules

2.3 Positioned Elements Required

3 Stacking Context

3.1 Creating Stacking Contexts

3.2 Stacking Context Impact

4 Z-Index in Practice

4.1 Simple Layering

4.2 UI Component Layering

5 Real-World Examples

5.1 Example 1: Modal with Overlay

5.2 Example 2: Navigation with Dropdowns

5.3 Example 3: Multiple Overlays

5.4 Example 4: Floating Elements

5.5 Example 5: Sticky with Fixed Header

6 Common Z-Index Mistakes

6.1 Mistake 1: Z-Index on Static Elements

6.2 Mistake 2: Not Understanding Stacking Contexts

6.3 Mistake 3: Stacking Context from Opacity

6.4 Mistake 4: Competing Stacking Contexts

6.5 Mistake 5: Arbitrary High Values

7 Z-Index Management Strategies

7.1 CSS Variables Approach

7.2 Layering Strategy

7.3 Naming Convention

7.4 Documentation

8 Advanced Techniques

8.1 Avoiding Stacking Context Creation

8.2 Debugging Z-Index Issues

8.3 Responsive Z-Index

9 Best Practices

10 Conclusion

11 Quick Reference

1 Introduction

The **z-index** property controls the vertical stacking order of elements in CSS, determining which elements appear on top when they overlap. However, z-index is often misunderstood because it only works with positioned elements and operates within stacking contexts. Understanding stacking contexts is crucial for mastering z-index and avoiding frustrating bugs where higher z-index values seem to have no effect.

Z-index becomes essential when building complex interfaces with multiple overlapping elements like modals, dropdowns, tooltips, and notifications. Mastering z-index requires understanding not just the property itself, but the underlying stacking context rules that govern how elements layer.

1.1 Why Z-Index Matters

- Layering Control: Manage which elements appear on top
- UI Components: Overlays, modals, dropdowns, tooltips
- Visual Hierarchy: Communicate importance through layering
- Complex Layouts: Handle multiple overlapping elements
- Interactive Effects: Create sophisticated interactions
- Responsive Design: Ensure correct layering across viewports

2 Understanding Z-Index Fundamentals

2.1 Basic Syntax

```
/* z-index values */
z-index: auto;           /* Default value */
z-index: 1;              /* Positive integer */
z-index: 100;            /* Higher appears on top */
z-index: -1;             /* Negative integer (behind) */
z-index: 0;              /* Same as auto for stacking */
```

2.2 Key Rules

1. Only works with positioned elements: z-index has no effect on static elements
2. Higher values appear on top: 100 appears above 50
3. Creates stacking context: Certain properties isolate z-index
4. Stacking context is local: Z-index only meaningful within context
5. Auto value is 0: Elements with auto have same stacking as normal flow

2.3 Positioned Elements Required

```
/* z-index works here */
.positioned {
  position: relative;
  z-index: 10;
}

.positioned-absolute {
  position: absolute;
  z-index: 20;
}

.positioned-fixed {
  position: fixed;
  z-index: 30;
}

/* z-index ignored on static elements */
.static {
  position: static;
  z-index: 9999; /* Has no effect */
}
```

3 Stacking Context

A stacking context is a three-dimensional conceptualization of elements on the z-axis. Elements within a stacking context are stacked together, and the context itself has a z-index relative to other contexts.

3.1 Creating Stacking Contexts

Elements that create new stacking contexts:

```
/* 1. Root element */
html { /* Creates root stacking context */ }

/* 2. position with z-index (if not auto) */
.element {
  position: relative;
  z-index: 1;
}

/* 3. opacity less than 1 */
.transparent {
  opacity: 0.5;
}

/* 4. filter property */
.filtered {
  filter: blur(5px);
}

/* 5. transform property */
.transformed {
  transform: scale(1.1);
}

/* 6. mix-blend-mode */
.blended {
  mix-blend-mode: multiply;
}

/* 7. clip-path */
.clipped {
  clip-path: polygon(0 0, 100% 0, 50% 100%);
}

/* 8. perspective */
.perspective {
  perspective: 1000px;
}

/* 9. overflow: not visible */
.overflow {
  overflow: auto;
}

/* 10. isolation property */
.isolated {
  isolation: isolate;
}
```


3.2 Stacking Context Impact

```
/* Problem: z-index within contexts */
.parent {
  position: relative;
  z-index: 1;
}

.parent .child {
  position: relative;
  z-index: 9999;
  /* Still cannot go above z-index: 2 siblings */
}

.sibling {
  position: relative;
  z-index: 2; /* This appears above .child */
}
```

4 Z-Index in Practice

4.1 Simple Layering

```
/* Basic layering */
.background {
  position: absolute;
  z-index: 1;
}

.content {
  position: absolute;
  z-index: 2;
}

.overlay {
  position: absolute;
  z-index: 3;
}
```

4.2 UI Component Layering

```
:root {
  --z-dropdown: 100;
  --z-sticky: 200;
  --z-fixed-header: 300;
  --z-modal-backdrop: 400;
  --z-modal: 500;
  --z-tooltip: 600;
  --z-notification: 700;
}
```

5 Real-World Examples

5.1 Example 1: Modal with Overlay

```
<div class="modal-backdrop">
  <div class="modal">
    <h2>Modal Title</h2>
    <p>Content</p>
    <button>Close</button>
  </div>
</div>

.modal-backdrop {
  position: fixed;
  top: 0; left: 0;
  width: 100%; height: 100%;
  background: rgba(0, 0, 0, 0.5);
  z-index: 1000;
  display: flex;
  align-items: center;
  justify-content: center;
}

.modal {
  position: relative;
  background: white;
  border-radius: 8px;
  padding: 2rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.15);
  z-index: 1001;
  max-width: 500px;
}
```


5.2 Example 2: Navigation with Dropdowns

```
.header {
  position: fixed;
  top: 0;
  width: 100%;
  z-index: 100;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

.dropdown-menu {
  position: absolute;
  top: 100%;
  z-index: 101; /* Above header */
  opacity: 0;
  visibility: hidden;
  transition: all 0.3s;
}

.nav-item:hover .dropdown-menu {
  opacity: 1;
  visibility: visible;
}
```

5.3 Example 3: Multiple Overlays

```
/* Background layer */
.bg-layer {
  position: absolute;
  z-index: 1;
}

/* Content layer */
.content-layer {
  position: absolute;
  z-index: 2;
}

/* Overlay with opacity */
.overlay-layer {
  position: absolute;
  z-index: 3;
  opacity: 0.5;
}

/* Text above everything */
.text-layer {
  position: absolute;
  z-index: 4;
}
```

5.4 Example 4: Floating Elements

```
/* Fixed header */
.header {
  position: fixed;
  top: 0;
  z-index: 100;
}

/* Floating action button */
.fab {
  position: fixed;
  bottom: 30px; right: 30px;
  z-index: 99;
}

/* Notifications */
.notification {
  position: fixed;
  top: 70px;
  z-index: 101; /* Above header */
}
```

5.5 Example 5: Sticky with Fixed Header

```
/* Fixed header at top */
.header {
  position: fixed;
  top: 0;
  z-index: 100;
}

/* Sticky section below header */
.section-header {
  position: sticky;
  top: 60px; /* Below fixed header */
  z-index: 99;
}

body { margin: 0; }
```

6 Common Z-Index Mistakes

6.1 Mistake 1: Z-Index on Static Elements

Wrong (has no effect):

```
.element {  
  position: static;  
  z-index: 9999;  
}
```

Correct:

```
.element {  
  position: relative;  
  z-index: 9999;  
}
```

6.2 Mistake 2: Not Understanding Stacking Contexts

Problem:

```
.parent {  
  position: relative;  
  z-index: 1;  
}  
  
.parent .child {  
  z-index: 9999;  
  /* Still can't go above z-index: 2 sibling */  
}  
  
.sibling {  
  position: relative;  
  z-index: 2;  
}
```

Solution (move child outside):

```
.child {  
  position: fixed;  
  z-index: 9999;  
  /* Now appears above sibling */  
}
```

6.3 Mistake 3: Stacking Context from Opacity

Problem:

```
.parent {  
  opacity: 0.9;  
  /* Creates stacking context */  
}  
  
.parent .child {  
  z-index: 9999;  
  /* Trapped in parent's context */  
}
```

6.4 Mistake 4: Competing Stacking Contexts

Problem:

```
.modal {  
  position: fixed;  
  z-index: 1000;  
  transform: translateZ(0);  
}  
  
.tooltip {  
  position: absolute;  
  z-index: 9999;  
}
```

6.5 Mistake 5: Arbitrary High Values

Bad practice:

```
.element-a { z-index: 999999; }  
.element-b { z-index: 999998; }  
.element-c { z-index: 999997; }
```

Better practice:

```
:root {  
  --z-base: 100;  
  --z-modal: 200;  
  --z-tooltip: 300;  
}
```

7 Z-Index Management Strategies

7.1 CSS Variables Approach

```
:root {  
  /* Base layers */  
  --z-hide: -1;  
  --z-base: 0;  
  
  /* Content layers */  
  --z-content: 10;  
  --z-overlay: 20;  
  
  /* UI layers */  
  --z-dropdown: 100;  
  --z-sticky: 200;  
  --z-fixed: 300;  
  
  /* Modal/Dialog layers */  
  --z-modal-backdrop: 1000;  
  --z-modal: 1001;  
  
  /* Top layers */  
  --z-tooltip: 2000;  
  --z-notification: 2001;  
}
```

7.2 Layering Strategy

Layer 7: Notifications, Alerts (z-index: 2001)

Layer 6: Tooltips (z-index: 2000)

Layer 5: Modals (z-index: 1001)

Layer 4: Modal Backdrops (z-index: 1000)

Layer 3: Fixed/Sticky Elements (z-index: 300)

Layer 2: Dropdowns (z-index: 100)

Layer 1: Content (z-index: 10)

Layer 0: Base/Hidden (z-index: -1 to 0)

7.3 Naming Convention

```
:root {  
  --z-below: -1;  
  --z-normal: 0;  
  --z-content: 10;  
  --z-dropdown: 100;  
  --z-sticky-nav: 200;  
  --z-fixed-header: 300;  
  --z-dialog-backdrop: 1000;  
  --z-dialog: 1001;  
  --z-tooltip: 2000;
```

```
--z-notification: 2001;  
--z-debug: 9999;  
}
```

8 Advanced Techniques

8.1 Avoiding Stacking Context Creation

```
/* DON'T do this if you need z-index
   to work across contexts */
.element {
  opacity: 0.99;
  filter: brightness(1);
  transform: translateZ(0);
}

/* DO this instead */
.element {
  position: relative;
  z-index: 10;
}
```

8.2 Debugging Z-Index Issues

```
/* Temporary debugging */
[data-z-debug] {
  outline: 2px solid red;
  position: relative;
  z-index: 9999 !important;
}
```

8.3 Responsive Z-Index

```
/* Mobile */
@media (max-width: 768px) {
  :root {
    --z-mobile-menu: 1000;
    --z-mobile-modal: 1001;
  }
}

/* Desktop */
@media (min-width: 769px) {
  :root {
    --z-desktop-dropdown: 100;
    --z-desktop-modal: 1000;
  }
}
```


9 Best Practices

DO:

- Use CSS custom properties for z-index values
- Create a clear, documented hierarchy
- Use meaningful names
- Keep z-index values reasonable (avoid 999999)
- Understand stacking contexts
- Test on multiple browsers
- Document exceptions
- Use consistent intervals (100, 200, etc.)

DON'T:

- Use z-index on static elements
- Use arbitrary high values
- Mix positioned and unpositioned elements
- Create unnecessary stacking contexts
- Ignore browser DevTools
- Use z-index as a layout tool
- Leave z-index undocumented
- Use !important with z-index

10 Conclusion

Mastering z-index requires understanding both the property and stacking contexts. Here are the key takeaways:

1. Z-index only works on positioned elements — not on static elements
2. Stacking contexts isolate z-index — a z-index: 9999 child cannot escape a z-index: 1 parent
3. Many properties create stacking contexts — opacity, transform, filter, etc.
4. Use CSS variables — organize and document your z-index strategy
5. Keep it simple — avoid arbitrary high values and complex hierarchies
6. Test thoroughly — z-index bugs are often browser-specific

With these principles and strategies, you can manage complex layering in your applications with confidence.

11 Quick Reference

```
/* Complete z-index template */
:root {
  /* Hidden/Below */
  --z-hide: -1;
  --z-base: 0;

  /* Content */
  --z-content: 10;

  /* UI Components */
  --z-dropdown: 100;
  --z-sticky: 200;
  --z-fixed-header: 300;

  /* Modals */
  --z-modal-backdrop: 1000;
  --z-modal: 1001;
  --z-modal-content: 1002;

  /* Top Layers */
  --z-tooltip: 2000;
  --z-notification: 2001;

  /* Emergency */
  --z-debug: 9999;
}
```

Happy layering!